##BFS && DFS:

```
#include<bits/stdc++.h>

using namespace std;

int node,edge;

int visited_array[1000];


vector<int>a[1000];


void BFS(int src)

{

   queue<int>b;

   b.push(src);

   visited_array[src]=1;

   while(!b.empty())

   {

      int head=b.front();

      cout<<head<<" ";

      b.pop();

      for(int child:a[head])

      {

         if(visited_array[child]==0)

         {

            visited_array[child]=1;

            b.push(child);

         }

      }

   }


}
```

```
void DFS(int node)
{
   cout<<node<<" ";
   visited_array[node]=1;
   for(int child:a[node])
   {
      if(visited_array[child]==0)
      {
         DFS(child);
      }
   }
}

int main()
{
   cout<<"Enter the number of Nodes and Edges"<<endl;
   cin>>node>>edge;
   cout<<"Enter the value"<<endl;
   for(int i=0; i<edge; i++)
   {
      int u,v;
      cin>>u>>v;
      a[u].push_back(v);
      a[v].push_back(u);
   }
    int b,d;
    cout<<"Bfs and dfs:";
    cin>>b;
```

```
    cout<<"BFS:";

    BFS(b);

    cout<<endl;

     memset(visited_array, 0, sizeof(visited_array));

    cout<<"DFS:";

    DFS(b);

  cout<<"";

    return 0;

}
```

##Articulation point

Code:

```
#include <bits/stdc++.h>

using namespace std;

const int N = 1e5 + 10;

vector<char> adj[N], ans, vertex;

vector<int> tin(N), low(N);

bool vis[N], mark[N];

int timer = 1;

// User function Template for C++


void dfs(int node, int parent)

{

    vis[node] = true;

    tin[node] = low[node] = timer;

    timer++;

    int child = 0;
```

```
    for (auto it : adj[node])
    {
        if (it == parent)
            continue;
        if (!vis[it])
        {
            dfs(it, node);
            low[node] = min(low[node], low[it]);
            if (low[it] >= tin[node] && parent != -1)
            {
                mark[node] = true;
            }
            child++;
        }
        else
        {
            low[node] = min(low[node], tin[it]);
        }
    }
    if (child > 1 && parent == -1)
    {
        mark[node] = true;
    }
}
void articulationPoints()
{
    for (auto it : vertex)
    {
        if (!vis[it])
```

```
        {
            dfs(it, -1);
        }
    }

    for (auto it : vertex)
    {
        if (mark[it])
        {
            ans.push_back(it);
        }
    }
    if (ans.size() == 0)
        cout << "No Articulation Point" << endl;
    else
    {
        cout << "Articulation Point: ";
        for (auto ap : ans)
        {
            cout << ap << " ";
        }
        cout << endl;
    }
}
int main()
{
    int v;
    cout << "Enter the number of nodes: ";
    cin >> v;
```

```
    cout << "Enter the nodes: ";
    for (int i = 0; i < v; i++)
    {
        char x;
        cin >> x;
        vertex.push_back(x);
    }
    int n;
    cout << "Enter the number of edges: ";
    cin >> n;
    cout << "Enter adjacent list: " << endl;
    for (int i = 0; i < n; i++)
    {
        char u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    articulationPoints();

    return 0;
}
```

######## 0/1 knapshake problem

```
#include<stdio.h>
#include<conio.h>
#include<iostream>

using namespace std;
```

```
int max(int a, int b)
{
   return (a > b) ? a : b;
}

int knapSack(int W, int wt[], int val[], int n)
{

   if (n == 0 || W == 0)
      return 0;


   if (wt[n - 1] > W)
      return knapSack(W, wt, val, n - 1);


   else
      return max(val[n - 1] + knapSack(W - wt[n - 1], wt, val, n - 1),
            knapSack(W, wt, val, n - 1));
}


int main()
{
   cout << "Enter the number of items in a Knapsack:";
   int n, W;
   cin >> n;
   int val[n], wt[n];
   for (int i = 0; i < n; i++)
```

```
    {
        cout << "Enter value and weight for item " << i << ":";
        cin >> val[i];
        cin >> wt[i];
    }



    cout << "Enter the capacity of knapsack";
    cin >> W;
    cout << knapSack(W, wt, val, n);


    return 0;
}
```

strongly connected components:

```
#include <bits/stdc++.h>
using namespace std;
vector<int> arr1[51];
vector<int> arr2[51];
int visited[51];
int visited2[51];
```

```
stack<int> st;
int r;
void DFS(int node){

    visited[node]=1;
    for(int child: arr1[node]){

        if(visited[child]!=1){
            DFS(child);

        }
    }
    st.push(node);
}
void DFS2(int node){
    cout<<node<<" ";
    visited2[node]=1;
    for(int child: arr2[node]){
    if(visited2[child]!=1){
            DFS2(child);

        }
    }

}
int main(){

    int n,m,x,y,a;
    cout<<"Enter number of vertex and number of edge"<<endl;
```

```
    cin>>n>>m;
    while(m--){
        cout<<"enter vertex having edge in ----> this way"<<endl;
        cin>>x>>y;
        arr1[x].push_back(y);
        arr2[y].push_back(x);
    }
    cout<<"enter starting node"<<endl;
        cin>>a;
    DFS(a);
    cout<<" strongly connected component "<<endl;
    while(!st.empty()){
        int k=st.top();
        st.pop();
        if(visited2[k]!=1){
            r++;
            cout<<endl;
            DFS2(k);
        }
    }
    cout<<endl;
    cout<<"Total number of strongly connected component = "<<r<<" follows above "<<endl;
    return 0;
}
```

###Dijksta source shortest path

```
#include <bits/stdc++.h>
```

```
using namespace std;

class Graph

{

public:

   vector<int> dijkstra(vector<vector<int>> &vec, int vertices, int edges, int source)

   {

      unordered_map<int, list<pair<int, int>>> adj;

      for (int i = 0; i < edges; i++)

      {

         int u = vec[i][0];

         int v = vec[i][1];

         int w = vec[i][2];

         adj[u].push_back(make_pair(v, w));

         adj[v].push_back(make_pair(u, w));

      }


      vector<int> dist(vertices, INT_MAX);

      set<pair<int, int>> st;

      dist[source] = 0;

      st.insert(make_pair(0, source));

      while (!st.empty())

      {

         auto top = *(st.begin());

         int nodeDistance = top.first;

         int topNode = top.second;

         st.erase(st.begin());


         for (auto neighbour : adj[topNode])

         {
```

```
            if (nodeDistance + neighbour.second < dist[neighbour.first])
            {
                auto record = st.find(make_pair(dist[neighbour.first], neighbour.first));
                if (record != st.end())
                {
                    st.erase(record);
                }
                dist[neighbour.first] = nodeDistance + neighbour.second;
                st.insert(make_pair(dist[neighbour.first], neighbour.first));
            }
        }
    }

    return dist;
  }
};

int main()
{
  Graph g;
  int vertices, edges;
  cout << "Enter the number of vertices: ";
  cin >> vertices;
  cout << "Enter the number of edges: ";
  cin >> edges;
  vector<vector<int>> edgeList(edges, vector<int>(3, 0));
  cout << "Enter the edges (u v w) - space-separated:" << endl;
  for (int i = 0; i < edges; i++)
  {
```

```
        cin >> edgeList[i][0] >> edgeList[i][1] >> edgeList[i][2];
    }
    int source;
    cout << "Enter the source node: ";
    cin >> source;
    vector<int> result = g.dijkstra(edgeList, vertices, edges, source);
    for (int i = 0; i < vertices; i++)
    {
        cout << "Distance from source to node " << i << ": " << result[i] << endl;
    }
  return 0;
}
```

##LCS (longest common sub)

```
#include<bits/stdc++.h>
using namespace std;
int main() {
    string s1, s2;
    cin >> s1 >> s2;
    int i, j, n = s1.size(), m = s2.size(), left, up, left_up;
    vector< vector<int> > dp(n+1, vector<int>(m+1, 0));
    for(i=1; i<=n; i++) {
        for(j=1; j<=m; j++) {
            left = dp[i][j-1];
            up = dp[i-1][j];
```

```
        if(s1[i-1] == s2[j-1]) left_up = dp[i-1][j-1]+1;
        else{
            left_up = INT_MIN;
        }
        if(left_up > left && left_up > up ) {
            dp[i][j] = left_up;
        }
        else if(up > left) {
            dp[i][j] = up;
        }
        else{
            dp[i][j] = left;
        }
    }
}
string lcs;
i = n;
j = m;
while(i>0 && j>0) {
    up = dp[i-1][j];
    left = dp[i][j-1];
    left_up = dp[i-1][j-1];
    if(up == dp[i][j]) {
        i--;
    }
    else if(left == dp[i][j]) {
        j--;
    }
    else{
```

```
            lcs.push_back(s1[i-1]);
            i--;
            j--;
        }
    }
    reverse(lcs.begin(), lcs.end());
    cout << "Length of lcs is: " << dp[n][m] << "\n" << "LCS: " << lcs << "\n";
}
```